Ollama

R&D

IA Locale — LLM sur Infrastructure Propre

En bref

Ollama est une plateforme open-source qui permet d'exécuter des modèles de langage (LLM) localement, sans aucune connexion à un service cloud. Les données ne quittent jamais le serveur. Idéal pour les clients sensibles (notaires, médecins, cabinets RH, défense) qui ne peuvent pas utiliser OpenAI ou Azure OpenAI. Phase R&D : les performances varient selon les modèles et le GPU disponible.

Point clé

Ollama nécessite un **VPS avec GPU** pour des performances acceptables (*Mistral 7B* en 5–10s/réponse). Sans GPU, les modèles légers (Phi-3, Gemma-2B) restent utilisables pour des tâches simples.

1 Ce que voit le client

Ce que voit et ressent le client

« J'utilise une IA pour analyser mes données financières et mes dossiers juridiques sensibles. Aucune donnée ne sort de mon réseau. Pas d'abonnement OpenAI, pas de cloud américain. Mon DPO et mon avocat ont validé sans réserve. »

Gains mesurables :

- **Confidentialité totale** : aucune donnée transmise à des tiers
- Idéal secteurs sensibles : médical, juridique, notariat, défense, RH
- Coût fixe (VPS GPU) vs coût variable (API par token)
- Pas de dépendance à un fournisseur cloud pour les fonctions IA critiques

2 Installation technique

Option A — VPS CPU uniquement (performances réduites) :

Listing 1 – docker-compose.yml — Ollama (CPU)

```yaml
version: "3.8"
services:
  ollama:
    image: ollama/ollama:latest
    restart: always
    volumes:
      - ollama_models:/root/.ollama
    ports:
      - "11434:11434"   # API interne uniquement (pas de Traefik ici)

  open-webui:
    image: ghcr.io/open-webui/open-webui:main
    restart: always
    depends_on:
      - ollama
    environment:
      OLLAMA_BASE_URL: http://ollama:11434
    volumes:
      - open_webui_data:/app/backend/data
    labels:
```

```
      - "traefik.enable=true"
      - "traefik.http.routers.openwebui.rule=Host(`ai-local.votredomaine.com`)"
      - "traefik.http.routers.openwebui.tls.certresolver=letsencrypt"

volumes:
  ollama_models:
  open_webui_data:
```

Option B — VPS GPU (Hetzner GX2 ou équivalent) :

Listing 2 – docker-compose.yml — Ollama (GPU NVIDIA)

```
services:
  ollama:
    image: ollama/ollama:latest
    restart: always
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: 1
              capabilities: [gpu]
    volumes:
      - ollama_models:/root/.ollama
```

Listing 3 – Télécharger et lancer les modèles

```
# Modèles recommandés selon les ressources disponibles

# CPU uniquement (RAM 8 GB+)
docker exec ollama ollama pull phi3:mini        # 2.3 GB - tâches simples
docker exec ollama ollama pull gemma2:2b        # 1.6 GB - très léger

# GPU 8 GB VRAM
docker exec ollama ollama pull mistral:7b       # 4.1 GB - polyvalent, excellent
docker exec ollama ollama pull llama3.1:8b      # 4.7 GB - très bon en français

# GPU 24 GB VRAM
docker exec ollama ollama pull mixtral:8x7b     # 26 GB - niveau GPT-3.5

# Vérifier les modèles disponibles
docker exec ollama ollama list

# Tester un modèle
docker exec ollama ollama run mistral "Résume ce texte en 3 points :"
```

3 Configuration de base

Connexion Flowise → Ollama (RAG local) :

1. Flowise → Credentials → Add → **Ollama**
2. Base URL : `http://ollama:11434` (réseau Docker interne)
3. Model : `mistral:7b` (ou autre modèle téléchargé)
4. Embeddings locaux : utiliser `nomic-embed-text` (`ollama pull nomic-embed-text`)

Connexion n8n → Ollama :

```
# Node HTTP Request dans n8n
POST http://ollama:11434/api/chat
Content-Type: application/json

{
  "model": "mistral:7b",
```

```
  "messages": [
    {"role": "system", "content": "Tu es un assistant expert en droit français."},
    {"role": "user", "content": "{{ $json.question }}"}
  ],
  "stream": false
}
```

Modèles recommandés par usage :

Usage	Modèle	Ressources
Résumé/classification	mistral :7b	GPU 8GB
Analyse juridique	llama3.1 :8b	GPU 8GB
Tâches légères (prod)	phi3 :mini	CPU 8GB
Embeddings RAG	nomic-embed-text	CPU 4GB

4 Intégration dans la stack

Outil	Rôle avec Ollama
Flowise	RAG 100% local : assistant sur documents sans cloud
n8n	Appels API Ollama depuis les workflows (HTTP Request node)
Open WebUI	Interface chat utilisateur pour tester les modèles
Wazuh	Les logs Wazuh ne sortent pas du serveur pour être analysés par IA

5 Rendre invisible côté client

— Le client accède à l'assistant via Open WebUI ou Teams — pas besoin de connaître Ollama
— Positionner Ollama comme « votre IA privée interne » — argument fort sur la confidentialité
— Le surcoût GPU (30–80 €/mois) est justifié pour les secteurs à haute confidentialité
— En mode CPU : déconseillé en production au-delà de 10 requêtes/jour (trop lent)

Maintenance mensuelle

- ☐ Vérifier que le modèle principal répond correctement (test `ollama run`)
- ☐ Contrôler l'utilisation GPU/mémoire (`nvidia-smi` si GPU disponible)
- ☐ Mettre à jour Ollama et les modèles : `docker compose pull && ollama pull mistral:7b`
- ☐ Vérifier l'espace disque des modèles (`docker exec ollama ollama list`)
- ☐ Tester l'intégration Flowise → Ollama : poser une question de test